

Laporan Tugas Besar

IF2224 TEORI BAHASA FORMAL DAN OTOMATA

MILESTONE 4: INTERMEDIATE CODE & INTERPRETER

SEMESTER II / 2025–2026



Disusun oleh:

GTFOTBFO (HBB)

Nathanael Imandatua Manurung - 13524021

Nicholas Wise Saragih Sumbayak - 13524037

Kurt Mikhael Purba - 13524065

Rava Khoman Tuah Saragih - 13524107

LABORATORIUM ILMU DAN REKAYASA KOMPUTASI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

Daftar Isi

1	Pendahuluan	3
1.1	Latar Belakang	3
1.2	Rumusan Masalah	4
1.3	Gambaran Umum Sistem	4
1.4	Spesifikasi Umum	4
2	Dasar Teori	5
2.1	Pendefinisian Bahasa Pemrograman	5
2.1.1	Hierarki Bahasa Pemrograman	5
2.1.2	<i>Compiler</i> dan <i>Interpreter</i>	5
2.2	Teori Kompilator (<i>Compiler Theory</i>)	5
2.2.1	<i>Intermediate Code Generation</i>	6
2.2.2	<i>Stack Machine</i>	7
2.2.3	<i>Display</i>	7
2.3	Teori Bahasa Formal dan Otomata	8
2.3.1	Kaitannya dengan Milestone 1–3	8
2.3.2	<i>Three-Address Code</i> dan Mesin Abstrak	8
3	Perancangan	8
3.1	Perancangan <i>Intermediate Code Generator</i>	8
3.1.1	Struktur Data Instruksi	8
3.1.2	Algoritma <i>Code Generation</i>	8
3.2	Perancangan <i>Stack Machine</i>	9
3.3	Perancangan <i>Interpreter</i>	10
3.4	Perancangan Penanganan <i>Runtime Error</i>	10
4	Implementasi	11
4.1	Struktur Direktori Proyek	11
4.2	Implementasi <code>Instruction.hpp</code> dan <code>Instruction.cpp</code>	12
4.3	Implementasi <code>CodeGenerator</code>	12
4.3.1	Expression dan Assignment	12
4.3.2	Kontrol Alur (If, While, For, Repeat)	13
4.3.3	Subprogram (Procedure dan Function)	13
4.4	Implementasi <code>StackMachine</code>	14
4.5	Implementasi <code>Interpreter</code>	15
4.6	Pembagian Tugas	15
5	Pengujian	15
5.1	Pengujian Unit	15
5.2	Pengujian Integrasi	16
5.2.1	Kasus Uji 1: Basic	16
5.2.2	Kasus Uji 2: Aritmetika	17
5.2.3	Kasus Uji 3: If-Else	18
5.2.4	Kasus Uji 4: While	19
5.2.5	Kasus Uji 5: For Loop	20
5.2.6	Kasus Uji 6: Negatif – Division by Zero	21

6	Bonus Features	22
7	Kesimpulan	23
A	Referensi Repositori	23

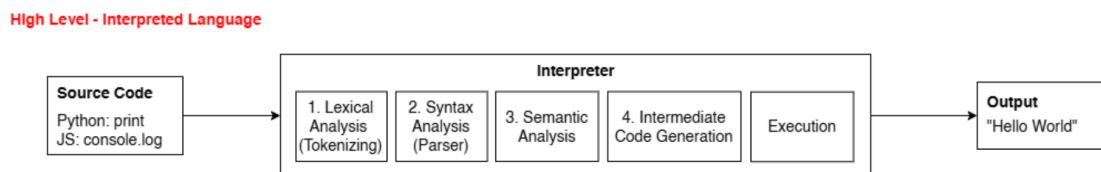
1 Pendahuluan

Pada tahap sebelumnya, telah berhasil dibangun sebuah *semantic analyzer* yang mampu mengkonversi *Parse Tree* menjadi *Decorated Abstract Syntax Tree* (Decorated AST) serta melakukan pengecekan semantik seperti kecocokan tipe data, *scope resolution*, dan deteksi *identifier* yang belum dideklarasikan. Meskipun *semantic analyzer* telah berfungsi dengan baik dalam memvalidasi makna program, tahapan kompilasi belum mencapai titik di mana program dapat dijalankan.

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik (*syntax analysis*), analisis semantik (*semantic analysis*), pembangkitan kode antara (*intermediate code generation*), dan eksekusi. Milestone keempat ini menyelesaikan dua tahap terakhir dalam rantai kompilasi hingga ke titik eksekusi program.

Pada milestone ini dibangun dua komponen utama:

1. **Intermediate Code Generator:** Komponen yang mentransformasi *Decorated AST* menjadi representasi kode antara berbentuk *Three-Address Code* (TAC). Setiap instruksi TAC terdiri dari nomor baris, opcode, level, dan operand. Kode antara ini menjadi jembatan antara representasi bahasa tingkat tinggi dan mesin eksekusi abstrak.
2. **Stack Machine Interpreter:** Komponen yang mengeksekusi instruksi-instruksi TAC menggunakan mesin *stack* dengan manajemen *frame* dan *display*. Interpreter bekerja dalam siklus *Fetch-Decode-Execute* untuk menjalankan setiap instruksi dan menghasilkan keluaran program.



Gambar 1: Pipeline kompilasi lengkap hingga Milestone 4

Sumber: Spesifikasi Milestone 4 Tugas Besar IF2224

1.1 Latar Belakang

Proses kompilasi suatu bahasa pemrograman terdiri atas beberapa tahap yang saling berkaitan, dimulai dari analisis leksikal (*lexical analysis*), analisis sintaktik, analisis semantik, hingga pembangkitan kode (*code generation*). Setelah *semantic analyzer* berhasil memvalidasi makna program dan menghasilkan *Decorated AST*, tahap selanjutnya adalah mengubah representasi tersebut menjadi instruksi-instruksi yang dapat dieksekusi oleh mesin.

Pada tugas besar mata kuliah IF2224 Teori Bahasa Formal dan Otomata, mahasiswa diminta untuk membangun sebuah *compiler* sederhana untuk bahasa pemrograman Arion, yaitu bahasa bertipe Pascal. Milestone keempat dan terakhir ini berfokus pada penggunaan *intermediate code generator* yang mengonversi *Decorated AST* menjadi instruksi

Three-Address Code (TAC), serta *interpreter* berbasis *stack machine* yang mengeksekusi instruksi-instruksi tersebut untuk menjalankan program.

1.2 Rumusan Masalah

Bagaimana merancang dan mengimplementasikan:

1. ***Intermediate Code Generator*** yang mampu mengonversi *Decorated AST* menjadi instruksi *Three-Address Code* (TAC) yang mencakup ekspresi, penugasan, kontrol alur (*if*, *while*, *for*, *repeat*), subprogram (*procedure/function*), dan I/O.
2. ***Stack Machine*** yang mampu mengelola memori stack, *frame* untuk setiap pemanggilan subprogram, *display* untuk resolusi *scope*, serta mendeteksi dan menangani kesalahan *runtime* seperti *stack overflow*, pembagian dengan nol, dan *jump* tidak valid.
3. ***Interpreter*** yang mampu mengeksekusi daftar instruksi TAC dalam siklus *Fetch-Decode-Execute* serta menjalankan seluruh operasi aritmetika, logika, relasional, dan I/O.

1.3 Gambaran Umum Sistem

Sistem yang dibangun pada Milestone 4 terdiri dari dua komponen utama:

1. ***Intermediate Code Generator*** (`src/intermediate/`): Menerima *Decorated AST* dan *symbol table* dari tahap analisis semantik, lalu menghasilkan daftar instruksi TAC.
2. ***Runtime Environment*** (`src/runtime/`): Terdiri dari *StackMachine* (manajemen stack dan frame) dan *Interpreter* (siklus eksekusi instruksi). Interpreter membaca instruksi TAC, mendekode opcode, dan mengeksekusi operasi yang sesuai pada stack.

Keluaran akhir sistem terdiri dari:

- ***Intermediate Code***: Daftar instruksi TAC yang dihasilkan dari proses *code generation*.
- ***Program Output***: Hasil eksekusi program yang dicetak ke `stdout` melalui instruksi `WRT` dan `WRTLN`.

1.4 Spesifikasi Umum

1. Intermediate Code Generation:

- Mengubah *Decorated AST* menjadi instruksi TAC dengan format `<line> <INSTR> <level> <operand>`.
- Mendukung 9 opcode: `INT`, `LIT`, `LOD`, `STO`, `CAL`, `JMP`, `JPC`, `OPR`, `RET`.
- Mendukung ekspresi, penugasan, I/O (`write`, `writeln`, `read`, `readln`).
- Mendukung kontrol alur: `if-then-else`, `while-do`, `for-to/downto`, `repeat-until`.

- Mendukung subprogram: prosedur dan fungsi dengan parameter.
- Manajemen *label* untuk *backpatching* instruksi lompatan.

2. Stack Machine & Interpreter:

- Manajemen stack dengan struktur *frame* per pemanggilan subprogram (SL, DL, RA, variabel lokal).
- Mekanisme *display* untuk resolusi *lexical level* (*static scope*).
- Siklus eksekusi *Fetch-Decode-Execute* untuk menjalankan instruksi TAC.
- 14 operasi OPR: NEG, ADD, SUB, MUL, DIV, MOD, EQL, NEQ, LSS, GEQ, GTR, LEQ, WRT, WRTLN.
- Penanganan kesalahan *runtime*: *stack overflow/underflow*, pembagian nol, *jump* tidak valid, akses di luar batas, dan korupsi stack.

2 Dasar Teori

2.1 Pendefinisian Bahasa Pemrograman

Bahasa Pemrograman adalah cara manusia untuk dapat memberikan perintah (*instruction*) kepada sebuah komputer. Setiap bahasa pemrograman memiliki sintaks (*syntax*) dan semantik (*semantics*). Sintaks mengatur bagaimana simbol-simbol dituliskan, sedangkan semantik mengatur makna dari setiap konstruksi bahasa. Pada tahap ini, setelah program dinyatakan benar secara sintaksis dan semantik, program perlu ditransformasi menjadi instruksi-instruksi yang dapat dieksekusi oleh mesin.

2.1.1 Hierarki Bahasa Pemrograman

Berdasarkan tingkat abstraksinya, bahasa pemrograman diklasifikasikan menjadi dua kategori utama:

- ***Low-Level Language* (Bahasa Tingkat Rendah):** Bahasa yang instruksinya mendekati arsitektur perangkat keras dan bahasa mesin.
- ***High-Level Language* (Bahasa Tingkat Tinggi):** Bahasa yang dirancang agar lebih mudah dibaca dan dipahami oleh manusia.

2.1.2 *Compiler* dan *Interpreter*

1. ***Compiler*:** Menerjemahkan seluruh *source code* menjadi kode mesin sebelum program dijalankan.
2. ***Interpreter*:** Memproses kode sumber baris demi baris secara langsung. Pada Milestone 4, interpreter yang dibangun bekerja pada level instruksi TAC, bukan pada kode sumber asli.

2.2 Teori Kompilator (*Compiler Theory*)

Proses kompilasi terdiri dari fase-fase berurutan:

1. ***Lexical Analysis***: Pemindai teks yang menghasilkan token.
2. ***Syntax Analysis***: Pemeriksa struktur sintaks yang menghasilkan Parse Tree.
3. ***Semantic Analysis***: Pemeriksa makna program yang menghasilkan Decorated AST.
4. ***Intermediate Code Generation***: Menghasilkan representasi kode perantara (TAC) untuk mesin abstrak.
5. ***Code Optimization***: Mengoptimalkan kode perantara (opsional, tidak diimplementasikan).
6. ***Target Code Generation***: Menghasilkan kode mesin (tidak diimplementasikan – dieksekusi langsung oleh interpreter).

2.2.1 *Intermediate Code Generation*

Intermediate Code Generation adalah fase keempat dari kompilator yang mentransformasi representasi semantik program (*Decorated AST*) menjadi bentuk perantara yang bersifat abstrak dan independen terhadap mesin target. Salah satu bentuk representasi antara yang umum digunakan adalah *Three-Address Code* (TAC).

TAC disebut *three-address code* karena setiap instruksi umumnya memiliki bentuk $x = y \text{ op } z$, yang melibatkan tiga alamat: dua operand sumber dan satu alamat tujuan. Dalam implementasi proyek ini, TAC direpresentasikan dalam format yang lebih sederhana:

<line> <OPCODE> <level> <operand>

Setiap instruksi terdiri dari:

- **line**: Nomor baris instruksi (0-based, diisi otomatis).
- **OPCODE**: Kode operasi (INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET).
- **level**: Tingkat *lexical level* untuk resolusi scope.
- **operand**: Nilai literal, alamat, nomor operasi, atau baris target lompatan.

Daftar Instruksi TAC:

Tabel 1: Daftar instruksi TAC

Opcode	Operasi	Semantik
INT m	Initiate Memory	Alokasi frame sebesar m word di stack
LIT v	Load Literal	Push nilai literal v ke stack
LOD 1 a	Load Value	Push value dari address a pada level 1
STO 1 a	Store Value	Pop stack lalu simpan ke address a pada level 1
CAL 1	Call	Panggil subprogram di instruction line 1
JMP 1	Unconditional Jump	IP = 1
JPC 1	Conditional Jump	Jika $\text{pop}() == 0$, IP = 1
OPR o	Operation	Eksekusi operasi o pada stack
RET	Return	Kembali dari subprogram

Operasi OPR (1–14):

Tabel 2: Daftar operasi OPR

No	Nama	Stack Effect
1	NEG	Pop $a \rightarrow \text{Push}(-a)$
2	ADD	Pop $b, a \rightarrow \text{Push}(a+b)$
3	SUB	Pop $b, a \rightarrow \text{Push}(a-b)$
4	MUL	Pop $b, a \rightarrow \text{Push}(a*b)$
5	DIV	Pop $b, a \rightarrow \text{Push}(a/b)$ – cek div by zero
6	MOD	Pop $b, a \rightarrow \text{Push}(a\%b)$
7	EQL	Push $(a == b ? 1 : 0)$
8	NEQ	Push $(a != b ? 1 : 0)$
9	LSS	Push $(a < b ? 1 : 0)$
10	GEQ	Push $(a >= b ? 1 : 0)$
11	GTR	Push $(a > b ? 1 : 0)$
12	LEQ	Push $(a <= b ? 1 : 0)$
13	WRT	Pop $\rightarrow \text{print}$
14	WRTLN	Pop $\rightarrow \text{print} + \text{newline}$

2.2.2 Stack Machine

Stack Machine adalah model mesin abstrak yang menggunakan struktur data *stack* sebagai media penyimpanan utama. Setiap subprogram memiliki *frame* sendiri di dalam stack. Layout frame adalah sebagai berikut:

Tabel 3: Layout frame stack

Address	Isi	Keterangan
0	Static Link	Display[level] frame induk
1	Dynamic Link	Base pointer frame pemanggil
2	Return Address	Instruction line tujuan setelah RET
3	Variabel/Parameter ke-1	
4	Variabel/Parameter ke-2	
...	...	

Untuk level 0 (global), static link = 0. Untuk fungsi/prosedur di level L, static link = display[L-1].

2.2.3 Display

Display adalah mekanisme untuk mengelola *static scope* dalam bahasa pemrograman dengan *lexical scoping* seperti Pascal. *Display* merupakan array yang menyimpan *base pointer* dari *frame* untuk setiap *lexical level*. Ketika interpreter perlu mengakses variabel dari *scope* induk, ia mengikuti *static link chain* dari frame saat ini hingga mencapai frame pada level yang diinginkan.

2.3 Teori Bahasa Formal dan Otomata

2.3.1 Kaitannya dengan Milestone 1–3

Milestone 4 tidak memperkenalkan konsep baru dalam Teori Bahasa Formal dan Otomata secara langsung. Konsep-konsep dari milestone sebelumnya – *Regular Expression* dan DFA (M1), *Context-Free Grammar* dan Recursive Descent (M2), *Attributed Grammar* dan Visitor Pattern (M3) – telah digunakan untuk membangun fondasi yang diperlukan. Milestone 4 menggunakan *Decorated AST* yang dihasilkan dari M3 sebagai masukan untuk menghasilkan kode antara.

2.3.2 Three-Address Code dan Mesin Abstrak

TAC adalah bentuk representasi antara yang berada di antara AST dan kode mesin. Dalam konteks teori bahasa formal, TAC dapat dipandang sebagai bahasa dari sebuah mesin abstrak (*abstract machine*) yang memiliki stack sebagai memori utamanya. Bahasa ini didefinisikan oleh sekumpulan instruksi (opcode) dan aturan transisi status mesin (semantik operasional).

Mesin abstrak stack dapat dimodelkan sebagai *finite-state machine* dengan state berupa (stack, ip, display). Setiap instruksi menyebabkan transisi dari satu state ke state berikutnya, membentuk jejak eksekusi (*execution trace*) program.

3 Perancangan

3.1 Perancangan *Intermediate Code Generator*

Code generator dirancang dengan pola desain *Visitor* (*ASTVisitor*). Setiap jenis node AST memiliki fungsi `visit()` yang menghasilkan instruksi TAC yang sesuai. *Code*

generator berjalan dalam satu lintasan (*single pass*) pada AST yang sudah dianotasi.

3.1.1 Struktur Data Instruksi

Setiap instruksi TAC direpresentasikan oleh struktur berikut:

Listing 1: Struktur Instruction

```

1 struct Instruction {
2     int line;           // instruction index (0-based, auto-set)
3     Opcode opcode;     // INT, LIT, LOD, STO, CAL, JMP, JPC, OPR
                        // , RET
4     int level;         // static nesting level
5     int operand;       // literal value, address, operation
                        // number, etc.
6 };

```

3.1.2 Algoritma *Code Generation*

Berikut adalah algoritma *code generation* untuk setiap node AST:

- **ProgramNode:** Hitung `frameSize` dari `btabs`, emit INT, proses deklarasi, proses body, emit RET.
- **AssignNode:** `visit(value)`, emit STO dengan `level` dan `adr` dari target.
- **BinaryOpNode:** `visit(left)`, `visit(right)`, emit OPR dengan nomor operasi sesuai operator.
- **UnaryOpNode:** `visit(operand)`, emit OPR NEG atau NOT.
- **LiteralNode:** Emit LIT dengan nilai literal.
- **VariableNode:** Emit LOD dengan `level` dan `adr` dari symbol table.
- **IfNode:** Evaluasi kondisi, JPC ke else-label, then-branch, JMP ke end-label, else-branch.
- **WhileNode:** Place start-label, kondisi, JPC ke end-label, body, JMP ke start-label.
- **RepeatNode:** Place start-label, statements, kondisi, JPC ke start-label.
- **ForNode:** Init, STO, start-label, check condition (LEQ/GEQ), JPC, body, increment/decrement, JMP.
- **ProcDeclNode:** Catat `adr`, INT, lokal, body, RET.
- **FuncDeclNode:** Sama dengan ProcDeclNode.
- **ProcCallNode:** Untuk I/O: push argumen, OPR WRT/WRTLN. Untuk user-defined: push argumen, CAL.

3.2 Perancangan *Stack Machine*

Stack machine dirancang dengan operasi-operasi berikut:

- `push(value)`: Push nilai ke stack.
- `pop()`: Pop nilai dari stack (dengan deteksi underflow).
- `pushFrame(frameSize, staticLink, returnAddr)`: Alokasi frame baru dengan SL, DL, RA, dan inisialisasi variabel lokal.
- `popFrame()`: Pop seluruh frame dan kembalikan BP.
- `store(level, addr, value)`: Simpan value pada address di frame level tertentu melalui *static link chain*.
- `load(level, addr)`: Load value dari address di frame level tertentu.
- `setDisplay/getDisplay`: Manajemen display untuk scope.

3.3 Perancangan *Interpreter*

Interpreter dirancang dengan siklus *Fetch-Decode-Execute*:

Listing 2: Siklus utama interpreter

```

1  void Interpreter::execute(const vector<Instruction> &
   instructions) {
2      ip = 0;
3      while (ip >= 0 && ip < instructions.size()) {
4          const Instruction &inst = instructions[ip];
5          try {
6              switch (inst.opcode) {
7                  case INT: handleINT(inst); break;
8                  case LIT: handleLIT(inst); break;
9                  // ... semua opcode
10             }
11         } catch (const RuntimeError &e) {
12             errors.push_back(e);
13             return;
14         }
15         ip++;
16     }
17 }
```

Setiap handler instruksi didefinisikan secara terpisah:

- **handleINT**: Push Frame baru dengan static link dari display.
- **handleLIT**: Push `inst.operand` ke stack.
- **handleLOD**: Load value dari address dan push ke stack.
- **handleSTO**: Pop stack dan store ke address.

- **handleJMP**: Set `ip = operand - 1`.
- **handleJPC**: Pop condition; jika 0, set `ip = operand - 1`.
- **handleCAL**: Simpan return address dan set `ip = operand - 1`.
- **handleRET**: Pop frame dan set `ip = returnAddress - 1`.
- **handleOPR**: Dispatch ke 14 operasi OPR sesuai nomor operand.

3.4 Perancangan Penanganan *Runtime Error*

Sistem dirancang untuk mendeteksi dan melaporkan *runtime error* tanpa menyebabkan *crash*:

- **RuntimeError**: Struktur yang menyimpan tipe error, pesan, dan nomor baris instruksi tempat error terjadi.
- **Stack Overflow**: Deteksi saat `pushFrame()` melebihi `maxFrames` (default 1000).
- **Stack Underflow**: Deteksi saat `pop()` pada stack kosong.
- **Division By Zero**: Deteksi pada OPR DIV dan MOD.
- **Invalid Jump**: Validasi target JMP dan JPC berada dalam rentang instruksi.
- **Index Out of Bounds**: Validasi address absolut pada `store()` dan `load()`.

4 Implementasi

Intermediate code generator dan *interpreter* pada compiler Arion diimplementasikan dalam bahasa C++20. Kedua komponen ini berada pada direktori `src/intermediate/` dan `src/runtime/`.

4.1 Struktur Direktori Proyek

Listing 3: Tata letak direktori proyek

```

1 HBB-Tubes-IF2224-2026/
2   src/
3       main.cpp                # Titik masuk program
4       driver_m4.cpp           # Driver khusus Milestone
5           4
6       lexical/                # Lexical analyzer (M1)
7       syntax/                 # Syntax analyzer (M2)
8       semantic/               # Semantic analyzer (M3)
9       intermediate/           # Code generator (M4 --
10          BARU)
11       Instruction.hpp/cpp      # Opcode enum +
12          Instruction struct
13       CodeGenerator.hpp/cpp    # Kelas CodeGenerator (
14          ASTVisitor)

```

```

11         runtime/                                # Runtime environment (M4
           -- BARU)
12         RuntimeError.hpp                         # Struktur RuntimeError
13         StackMachine.hpp/cpp                     # Kelas StackMachine
14         Interpreter.hpp/cpp                      # Kelas Interpreter
15     test/
16         milestone4/
17             input/                                # Berkas uji masukan M4
18                 pos_basic.txt
19                 pos_arith.txt
20                 pos_if_else.txt
21                 pos_while.txt
22                 pos_for.txt
23                 ...
24             output/                              # Keluaran yang
                diharapkan
25                 pos_basic.txt
26                 ...
27     doc/
28         m4/                                       # Laporan M4
29             main.tex
30             *.png                               # Screenshot hasil uji
31     Makefile                                     # Skrip kompilasi (
                diperbarui)

```

4.2 Implementasi Instruction.hpp dan Instruction.cpp

Berkas Instruction.hpp mendefinisikan:

- enum class Opcode: INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET.
- struct Instruction: line, opcode, level, operand.
- std::string toString() const: format "<line> <OPCODE> <level> <operand>".

Listing 4: Deklarasi Instruction

```

1  enum class Opcode {
2      INT, LIT, LOD, STO, CAL, JMP, JPC, OPR, RET
3  };
4
5  struct Instruction {
6      int line;
7      Opcode opcode;
8      int level;
9      int operand;
10
11      std::string toString() const;
12  };

```

4.3 Implementasi CodeGenerator

Kelas CodeGenerator mengimplementasikan ASTVisitor dan menyediakan fungsi generate() sebagai titik masuk.

4.3.1 Expression dan Assignment

Listing 5: Implementasi ekspresi dan assignment

```

1 void CodeGenerator::visit(LiteralNode &node) {
2     int value = 0;
3     try { value = stoi(node.value); }
4     catch (...) {
5         if (node.value == "true") value = 1;
6         else if (node.value == "false") value = 0;
7     }
8     emit(LIT, 0, value);
9 }
10
11 void CodeGenerator::visit(VariableNode &node) {
12     const TabEntry &entry = symTable.getTab(node.tabIndex);
13     emit(LOD, entry.lev, entry.adr);
14 }
15
16 void CodeGenerator::visit(BinaryOpNode &node) {
17     if (node.left) node.left->accept(*this);
18     if (node.right) node.right->accept(*this);
19     int opNum = opToCode(node.op);
20     if (opNum > 0) emit(OPR, 0, opNum);
21 }
22
23 void CodeGenerator::visit(AssignNode &node) {
24     if (node.value) node.value->accept(*this);
25     if (node.target && node.target->tabIndex >= 0) {
26         const TabEntry &entry = symTable.getTab(node.target->
27             tabIndex);
28         emit(STO, entry.lev, entry.adr);
29     }
30 }

```

4.3.2 Kontrol Alur (If, While, For, Repeat)

Listing 6: Implementasi IfNode dengan backpatching

```

1 void CodeGenerator::visit(IfNode &node) {
2     int elseLabel = newLabel();
3     int endLabel = newLabel();
4
5     if (node.condition) node.condition->accept(*this);
6     int jpcIdx = instructions.size();
7     emit(JPC, 0, 0); // backpatched

```

```

8     if (node.thenBranch) node.thenBranch->accept(*this);
9
10    int jmpIdx = -1;
11    if (node.elseBranch) {
12        jmpIdx = instructions.size();
13        emit(JMP, 0, 0); // backpatched
14    }
15
16    placeLabel(elseLabel);
17    backpatch(jpcIdx, getLabelLine(elseLabel));
18    if (node.elseBranch) node.elseBranch->accept(*this);
19
20    placeLabel(endLabel);
21    if (jmpIdx >= 0) backpatch(jmpIdx, getLabelLine(endLabel)
22                               );
23 }

```

4.3.3 Subprogram (Procedure dan Function)

Listing 7: Implementasi deklarasi prosedur

```

1 void CodeGenerator::visit(ProcDeclNode &node) {
2     TabEntry &entry = symTable.getTab(symTable.lookup(node.
3         name));
4     entry.adr = instructions.size(); // catat entry point
5
6     int blockIdx = symTable.getDisplay(entry.lev);
7     BTabEntry &block = symTable.getBTab(blockIdx);
8     emit(INT, entry.lev, 3 + block.psize + block.vsize);
9
10    for (auto &decl : node.localDeclarations) {
11        if (decl) decl->accept(*this);
12    }
13    if (node.body) node.body->accept(*this);
14    emit(RET, 0, 0);
15 }

```

4.4 Implementasi StackMachine

StackMachine mengelola memori stack dan frame:

Listing 8: Implementasi pushFrame dan popFrame

```

1 void StackMachine::pushFrame(int frameSize, int staticLink,
2     int returnAddr) {
3     if (currentFrames >= maxFrames) {
4         throw RuntimeError(STACK_OVERFLOW, "Stack overflow...");
5     }
6     push(staticLink); // SL
7     push(bp);         // DL
8 }

```

```

7     push(returnAddr); // RA
8     for (int i = 3; i < frameSize; i++) push(0); // lokal
9     bp = sp - frameSize + 1;
10    currentFrames++;
11 }
12
13 void StackMachine::popFrame() {
14     if (currentFrames <= 0) {
15         throw RuntimeError(STACK_CORRUPTION, "No frame to pop
16         ");
17     }
18     int dynLink = stack[bp + 1];
19     int frameSize = sp - bp + 1;
20     for (int i = 0; i < frameSize; i++) pop();
21     bp = dynLink;
22     currentFrames--;
23 }

```

4.5 Implementasi Interpreter

Interpreter memiliki siklus utama `execute()` dan handler untuk setiap jenis instruksi:

Listing 9: Implementasi OPR operations

```

1 void Interpreter::handleOPR(const Instruction &inst) {
2     switch (inst.operand) {
3         case 1: { // NEG
4             int a = stack.pop(); stack.push(-a); break;
5         }
6         case 2: { // ADD
7             int b = stack.pop(), a = stack.pop();
8             stack.push(a + b); break;
9         }
10        case 3: { // SUB
11            int b = stack.pop(), a = stack.pop();
12            stack.push(a - b); break;
13        }
14        // ... case 4--14 ...
15        case 13: { // WRT
16            int a = stack.pop(); cout << a; break;
17        }
18        case 14: { // WRTLN
19            int a = stack.pop(); cout << a << endl; break;
20        }
21        default:
22            throw RuntimeError(UNKNOWN_INSTRUCTION, "...");
23    }
24 }

```


4.6 Pembagian Tugas

Tabel 4: Pembagian tugas antar anggota kelompok

Role	Anggota	Tanggung Jawab
1	Nicholas Wise Saragih Sumbayak - 13524037	Instruction struct, Expression/Assign
2	Nathanael Imandutua Manurung - 13524021	Address assignment, Control flow & S
3	Kurt Mikhael Purba - 13524065	StackMachine, Interpreter core(INT/LIT/LOD
4	Rava Khoman Tuah Saragih - 13524107	OPR operations, driver, test cases

5 Pengujian

Pengujian dilakukan terhadap program dari dua sisi: pengujian unit (*unit test*) untuk komponen StackMachine dan Interpreter, serta pengujian integrasi (*integration test*) menggunakan driver M4 yang menjalankan pipeline kompilasi penuh.

5.1 Pengujian Unit

Pengujian unit untuk Role 3 (`test/test_role3.cpp`) mencakup:

- StackMachine push/pop dan deteksi underflow
- Frame push/pop dan akses memori
- Static link chain / frame bersarang
- Interpreter LIT + STO + LOD
- Interpreter JMP (unconditional jump)
- Interpreter JPC (conditional jump)
- Interpreter CAL + RET
- Invalid jump target
- Out-of-bounds memory access
- Stack overflow (max frames)
- Display management

5.2 Pengujian Integrasi

Pilih minimal 5 kasus uji dari tabel berikut. Untuk setiap kasus, sertakan **input**, **keluaran yang diharapkan**, **screenshot hasil**, dan **analisis**.

- Kasus uji positif: basic, arithmetic, if-else, while, for, mixed, nested.
- Kasus uji negatif: division by zero.
- Kasus uji batas: loop besar.

5.2.1 Kasus Uji 1: Basic

Berkas: pos_basic.txt

Masukan:

```
1 program Basic;  
2 var x: integer;  
3 begin  
4     x := 42;  
5     writeln(x);  
6 end.
```

Keluaran yang diharapkan:

=== Intermediate Code ===

```
0 INT 0 4  
1 LIT 0 42  
2 STO 0 3  
3 LOD 0 3  
4 OPR 0 14  
5 RET 0 0
```

=== Program Output ===

42



testcase_basic.png

Gambar 2: Hasil pengujian kasus uji basic

Analisis: *[Isi analisis di sini]*

5.2.2 Kasus Uji 2: Aritmetika

Berkas: pos_arith.txt

Masukan:

```
1 program Arith;  
2 var x: integer;  
3 begin  
4     x := 10 + 20 * 3;  
5     writeln(x);  
6 end.
```

Keluaran yang diharapkan:



testcase_arith.png

Gambar 3: Hasil pengujian kasus uji aritmetika

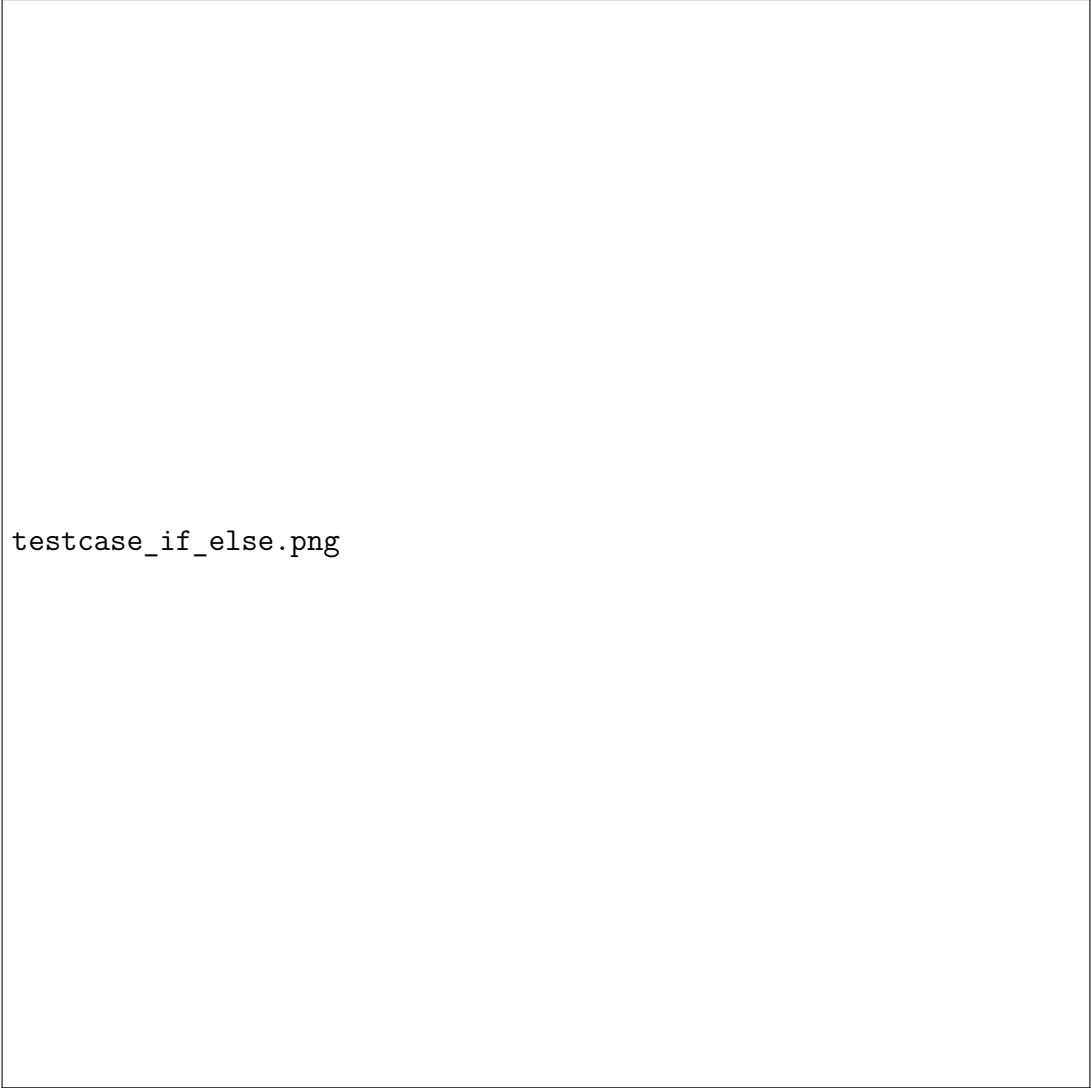
Analisis: *[Isi analisis di sini]*

5.2.3 Kasus Uji 3: If-Else

Berkas: pos_if_else.txt

Masukan: *[Isi source code di sini]*

Keluaran yang diharapkan: *[Isi output di sini]*



testcase_if_else.png

Gambar 4: Hasil pengujian kasus uji if-else

Analisis: *[Isi analisis di sini]*

5.2.4 Kasus Uji 4: While

Berkas: pos_while.txt

Masukan: *[Isi source code di sini]*

Keluaran yang diharapkan: *[Isi output di sini]*



testcase_while.png

Gambar 5: Hasil pengujian kasus uji while

Analisis: *[Isi analisis di sini]*

5.2.5 Kasus Uji 5: For Loop

Berkas: pos_for.txt

Masukan: *[Isi source code di sini]*

Keluaran yang diharapkan: *[Isi output di sini]*



testcase_for.png

Gambar 6: Hasil pengujian kasus uji for

Analisis: *[Isi analisis di sini]*

5.2.6 Kasus Uji 6: Negatif – Division by Zero

Berkas: neg_div_zero.txt

Masukan:

```
1 program DivZero;  
2 var x: integer;  
3 begin  
4     x := 10;  
5     writeln(x div 0);  
6 end.
```

Keluaran yang diharapkan:

Runtime error: Division by zero



testcase_divzero.png

Gambar 7: Hasil pengujian kasus uji division by zero

Analisis: *[Isi analisis di sini]*

6 Bonus Features

[Isi dengan bonus yang berhasil diimplementasikan, misalnya:]

1. **Stack Overflow** ... *[deskripsi]*
2. **Stack Underflow** ... *[deskripsi]*
3. **Invalid Jump Target** ... *[deskripsi]*
4. **Out-of-Bounds Access** ... *[deskripsi]*
5. **Numerical Overflow** ... *[deskripsi]*
6. **Stack Smashing** ... *[deskripsi]*

7 Kesimpulan

[Isi kesimpulan di sini]

A Referensi Repositori

Seluruh kode sumber proyek ini dapat diakses melalui repositori GitHub berikut:

- **URL Repository:** <https://github.com/nicholaswisee/HBB-Tubes-IF2224-2026>